

Software Security: Exploitation

Shuai Wang



香港科技大學

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Software bugs that can be potentially exploited

- The so-called “**vulnerability**”
 - Crash Causing Defects
 - Null pointer dereference
 - Use after free
 - Double free
 - Array indexing errors
 - Mismatched array new/delete
 - Stack overflow
 - Heap overflow
 - Return pointers to local variables
 - Logically inconsistent code
 - Uninitialized variables
 - Invalid use of negative values
 - Passing large parameters by value
 - Underallocations of dynamic data
 - Memory leaks
 - File handle leaks
 - Network resource leaks
 - Unused values
 - Unhandled return codes
 - Use of invalid iterators

But note that many of the (logic) bugs are not exploitable and therefore are not “**vulnerabilities**”

Common Software Vulnerabilities

- Buffer overflows ← this time
- Format string problems
- Integer overflows
- Use after free ← next time
- Type confusion
- Race conditions...

Attacks marked in red are the most popular ones.

Buffer Overflow

- Used in 1988's Morris Internet Worm
- Alphe One's "*Smashing The Stack For Fun And Profit*" in 1996 popularizes stack buffer overflows
 - Your reading article today..
- Still extremely common today
 - Over 50% of advisories published by CERT (computer security incident report team) are caused by various buffer overflows



Computer Memory

Your own memory may look like this:

wake up; have breakfast; need to
buy milk; turn off the lights; go to
class; that man has a strange shirt;
fall asleep; wake up

A web server's memory may look like this:

Bob requests main page; Atta wants
reply "Cat"; Li sets password to
"sup3rsekr1t"; Kate wants image
"derpy_cat"; Poe sets secret key; ...

Memory Read



Please reply "Cat"
(3 letters).

Please reply "Cat"
(5 letters).

Bob requests main page; Atta wants
reply "Cat"; Li sets password to
"sup3rsekr1t"; Kate wants image
"derpy_cat"; Poe sets secret key; ...

Memory

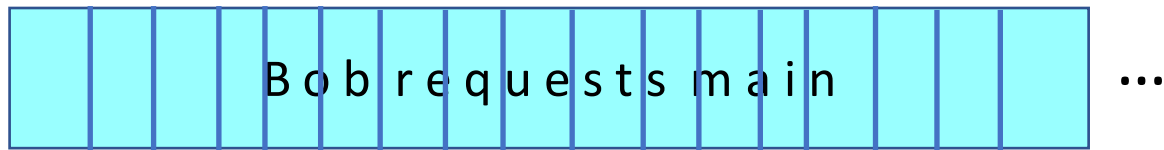
Cat



Cat";

Under the hood

```
char buffer[100];
```



- Buffer is a data storage area inside computer memory
- Intended to hold pre-defined amount of data
 - If more data is stored/required from it, it tampers the adjacent memory

Memory/Buffer Overread

Bob requests main page; Atta wants reply "Cat"; Li sets password to "sup3rsekr1t"; Kate wants image "derpy_cat"; Poe sets secret key; ...

Memory

Please reply "Cat"
(100 letters).

Cat"; Li sets password to
"sup3rsekr1t"; Kate wants
image "derpy_cat"; Poe
sets secret key; ...



Exploiting Memory For Fun and Profit

- All right, we talked about how buffer can be **overread**
 - The “Heartbleed” attack on OpenSSL
- Then, what about buffer **overwritten**?
 - Change the memory content?
 - What else?
 - Note that “**code**” can also be written in the memory... → executing arbitrary code!

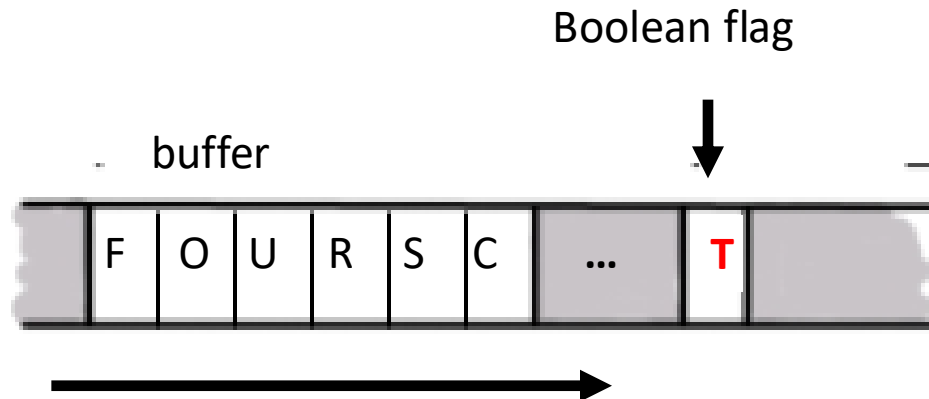


Buffer Overwrite Scenario

- Users enter data into a Web form
- Web form is sent to server
- Server writes data to array called buffer, **without checking length of input data**
- Data “overflows” buffer
 - Such overflow might enable an attack
 - If so, attack could be carried out by anyone with Internet access

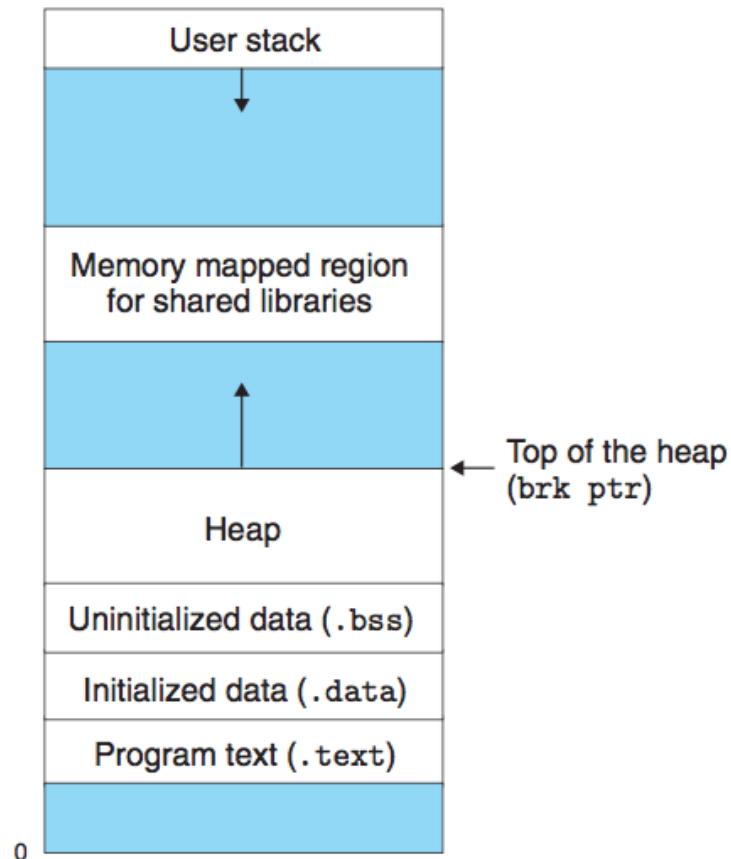
Simple Buffer Overflow

- Consider boolean flag to check password correct or not
- Buffer overflow could overwrite flag allowing anyone to authenticate

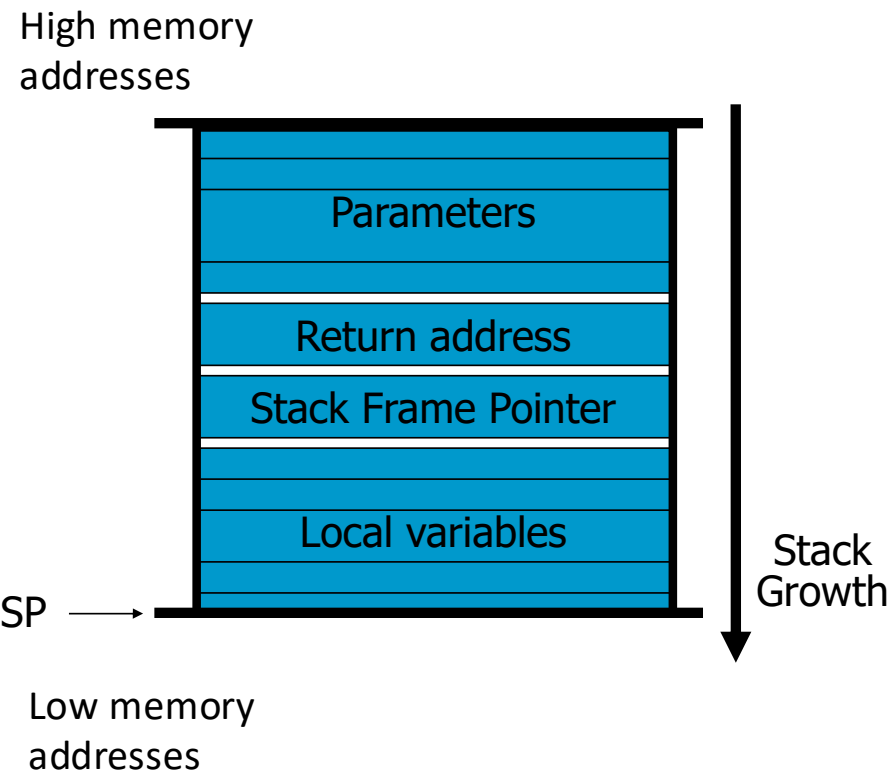


How is memory implemented?

- Linux standard memory layout for a process but Mac/Windows are very similar.



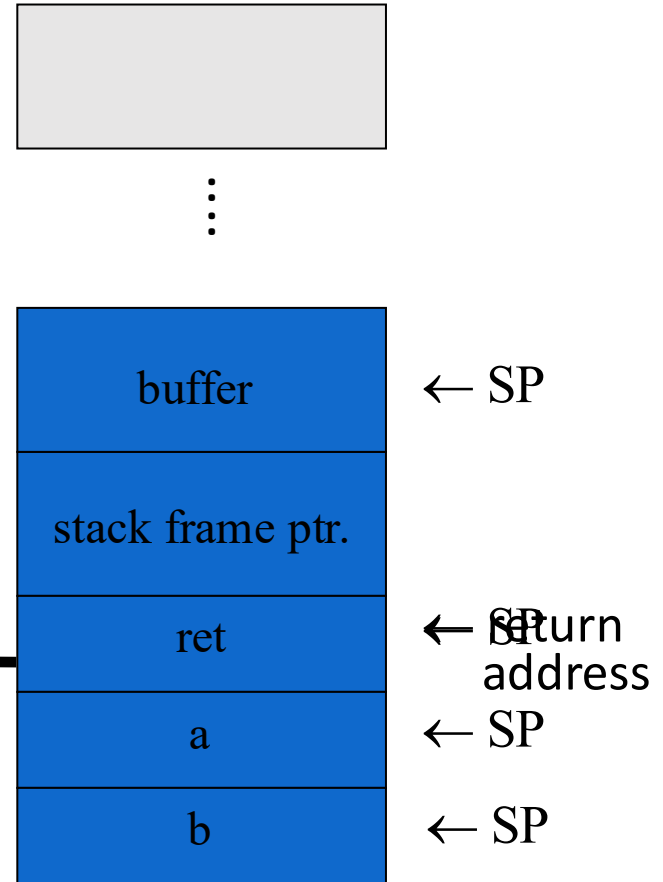
A Stack Frame



- A stack consists of logical stack frames that are pushed when calling a function and popped when returning.
- When a function is called, the return address, stack frame pointer and the variables are pushed on the stack.
- The stack grows from high address to low address.
 - When we overflow the buffer, the return address will be overwritten.

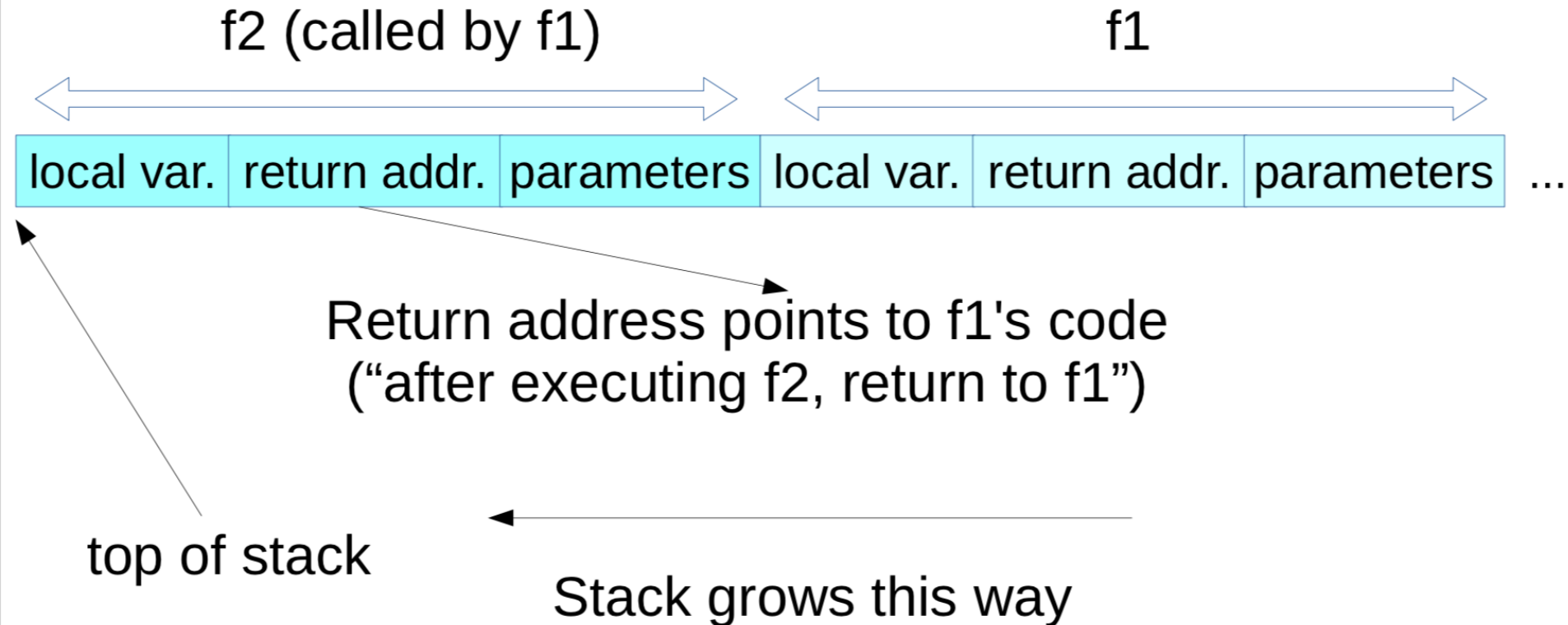
Stack Example

```
void func(int a, int b){  
    char buffer[10];  
    return;  
}  
void main(){  
    func(1,2);  
    int c = 1 + 1;  
}
```



A Sequence of Stack Frames

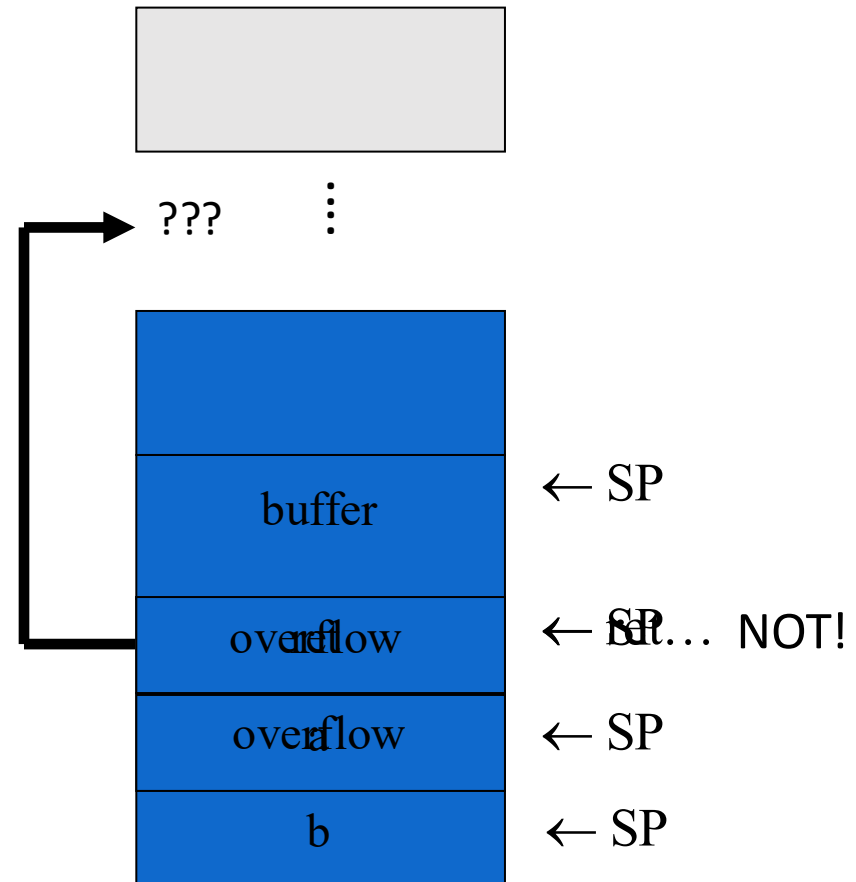
A simplified function stack (ignored stack frame pointer)!



Smashing the Stack

Ignored stack frame pointer since it does not affect our discussion!

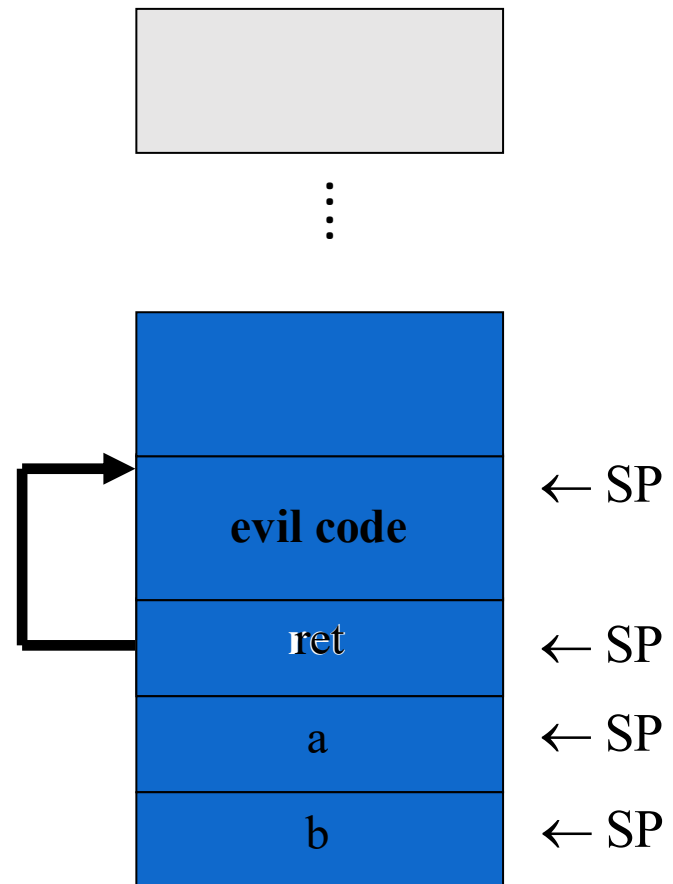
- ❑ What happens if buffer overflows?
- ❑ Program “returns” to wrong location
- ❑ A crash is likely



Smashing the Stack

Ignored stack frame pointer since it does not affect our discussion!

- ❑ An even better idea...
- ❑ **Code injection**
- ❑ Attacker can run arbitrary code on your machine
 - ❑ And even more ← later



Buffer Overflow Attack

- Also known as “stack smashing”, “buffer overrun”
 - Also known as a classic example of **code injection attack**, **control-flow hijacking attack**
- A buffer overflow must exist in the code
 - But not all buffer overflows are exploitable
- If exploitable, attacker can **inject code**
- Trial and error is likely required
 - Suppose we need to guess the address of **evil code**
 - But we only need one success anyway...
- Stack overflow is “attack of the decade” ...
 - Also heap overflow and so on.

Buffer Overflow Attack

- Buffer grows **backwards** on the stack and therefore **overwrites** the **return address**.
- Any thoughts on the mitigation...?
- What if buffer grows forwardly?
 - No, it does not solve the problem.
 - See my notes.

Unsafe C/C++ Lib Functions

`strcpy (char *dest, const char *src)`

`gets (char *s)`

`strcat (char *dest, const char *src)`

`scanf ( const char *format, ... )`

`sprintf (const char *format, ... )`

...

Buffer Overflow Attack due to Unsafe C/C++ Library Functions

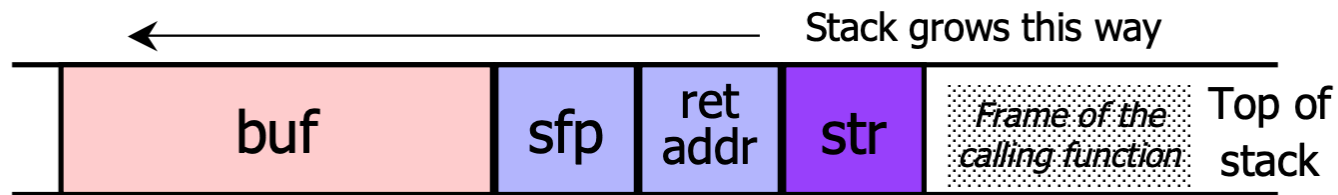
- Suppose Web server contains this function

```
void func(char *str) {  
    char buf[126];  
    strcpy(buf, str);  
}
```

Allocate local buffer
(126 bytes reserved on stack)

Copy argument into local buffer

- When this function is invoked, a new **frame** with local variables is pushed onto the stack



Buffer Overflow Attack due to Unsafe C/C++ Library Functions

- Memory pointed to by str is copied onto stack...

```
void func(char *str) {  
    char buf[126];  
    strcpy(buf, str);  
}
```

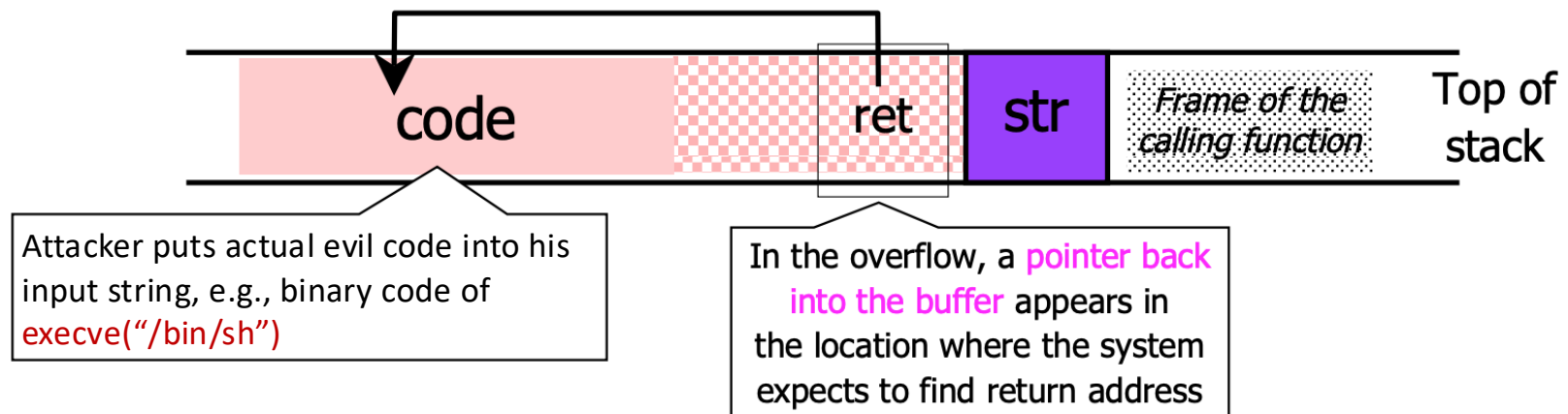
strcpy does NOT check whether the string at *str contains fewer than 126 characters

- If a string longer than 126 bytes is copied into buffer, it will overwrite adjacent stack locations



Buffer Overflow Attack due to Unsafe C/C++ Library Functions

- Suppose buffer contains attacker-created string
 - For example, `*str` contains a string received from the network as input to some network service daemon



- When function exits, code in the buffer will be executed, giving attacker a shell
 - **Root shell** if the victim program is `setuid root`

Buffer Overflow Attack due to Unsafe C/C++ Library Functions

```
void input_username(...) {  
    char username[16];  
    printf("Enter username:");  
    gets(username);  
    ...  
}
```

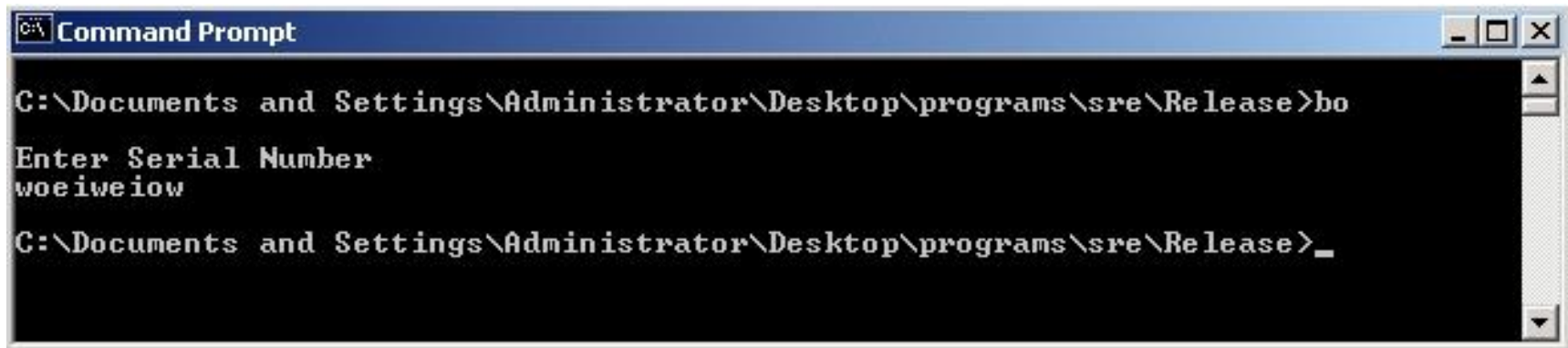
gets does not check bounds!

[] [1200]

...	username[16]	return addr.	Parameters
-----	--------------	--------------	------------

Stack Overflow Attack Example

- Suppose program asks for a serial number that you do not know
- Also, you do **not** have source code
- You only has the executable (exe)

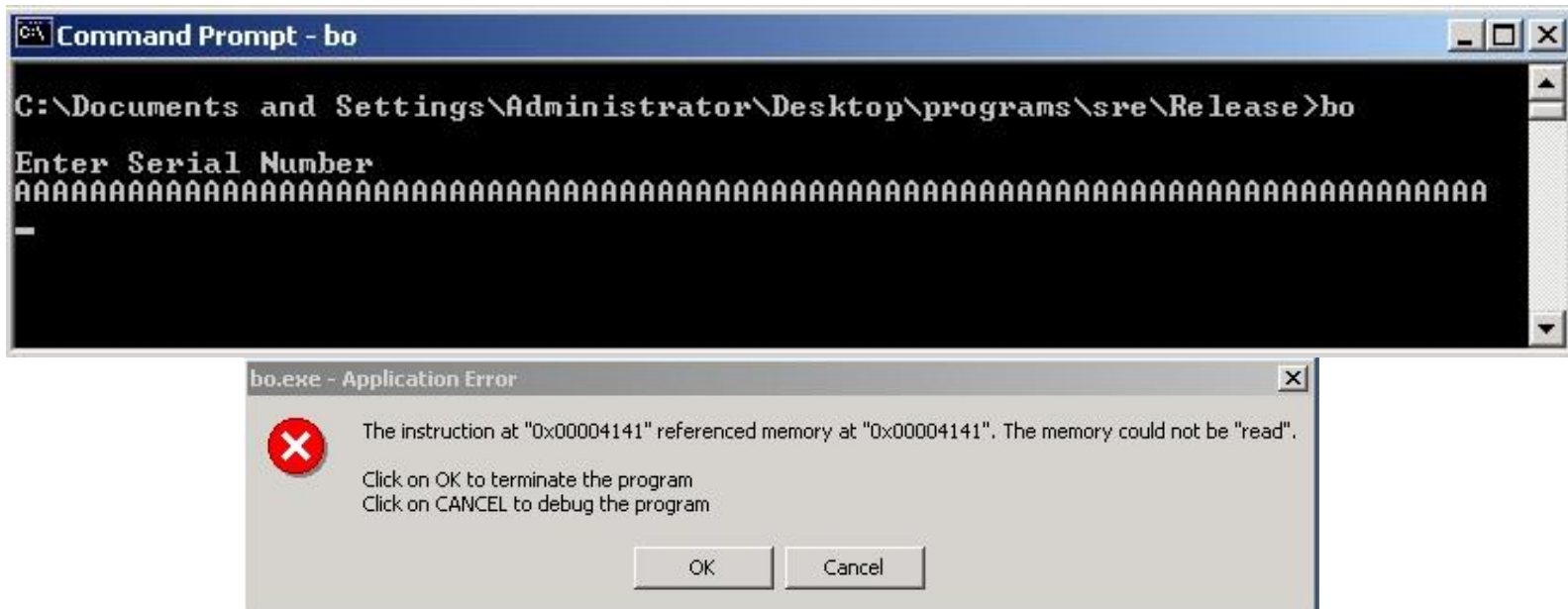


```
Command Prompt
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>bo
Enter Serial Number
woeiweiw
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>_
```

- ❑ Program quits on incorrect serial number

Buffer Overflow Present?

- By trial and error, attacker discovers apparent buffer overflow



- ❑ Note that 0x41 is ASCII for "A"
- ❑ Looks like **ret** overwritten by 2 bytes!

Disassemble Code

- Next, disassemble bo.exe to find

```
.text:00401000
.text:00401000
.text:00401003
.text:00401008
.text:0040100D
.text:00401011
.text:00401012
.text:00401017
.text:0040101C
.text:0040101E
.text:00401022
.text:00401027
.text:00401028
.text:0040102D
.text:00401030
.text:00401032
.text:00401034
.text:00401039
.text:0040103E

sub     esp, 1Ch
push    offset aEnterSerialNum ; "\nEnter Serial Number\n"
call    sub_40109F
lea     eax, [esp+20h+var_1C]
push    eax
push    offset aS          ; "%S"
call    sub_401088
push    8
lea     ecx, [esp+2Ch+var_1C]
push    offset aS123n456 ; "S123N456"
push    ecx
call    sub_401050
add     esp, 18h
test    eax, eax
jnz     short loc_401041
push    offset aSerialNumberIs ; "Serial number is correct.\n"
call    sub_40109F
add     esp, 4
```

- ❑ The goal is to exploit buffer overflow to jump to address 0x401034

Overflow Attack, Take 2

- Reverse the byte order to “4^P@a” and...



The screenshot shows a Windows Command Prompt window with the title bar 'C:\ Command Prompt'. The command prompt is at the directory 'C:\Documents and Settings\Administrator\Desktop\programs\sre\Release'. The user has entered 'bo' and the program has prompted for a serial number. The user has entered a long string of 'A's followed by '4^P'. The program has responded that the serial number is correct.

```
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>bo
Enter Serial Number
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA4^P
Serial number is correct.
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>_
```

- ❑ Success! We've bypassed serial number check by exploiting a buffer overflow
- ❑ What just happened?
 - Overwrote return address on the stack

Buffer Overflow

- Attacker did **not** require access to the source code
- Only tool used was a disassembler to determine address to jump to
- Find desired address by trial and error?
 - Necessary if attacker **does not have exe**
 - For example, a remote attack

Buffer Overflow Defense

- Never execute code on stack
 - W^X (write XOR Execute)
- Detect overflow
 - Use a **canary**
- Randomize stack
 - Address space layout randomization (**ASLR**)
- Don't use C/C++
 - Use **safe languages** (Java, C#)
- Use **safer C functions**
 - Additional checks on the buf length.

Marking stack as non-execute

- Basic stack exploit can be prevented by marking stack segment as non-executable.
 - W^X (write XOR Execute)
 - Support in Windows. Code patches exist for Linux.

Problems:

- Some software need writeable .text section or executable stack section
 - “self-modifying code”
 - Just-in-time compilation

Randomization: Motivations

- Buffer overflow exploits need to know the (virtual) address to which pass control
 - Address of attack code in the buffer
- Same address is used on many machines
- Idea: introduce **diversity**
 - Make stack addresses unpredictable and different from machine to machine

Address Space Layout Randomization (ASLR)

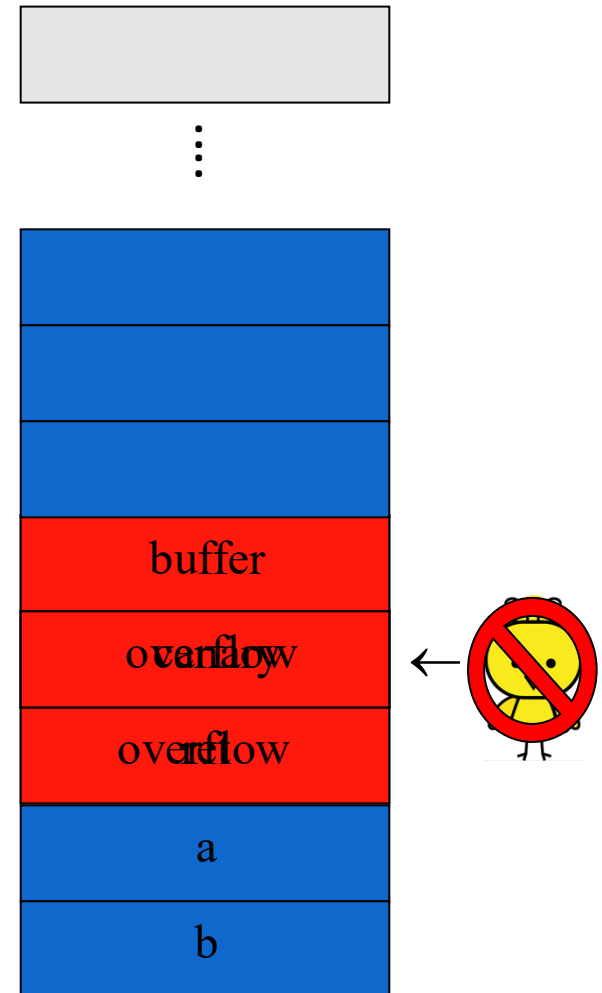
- Arranging the positions of key data areas randomly in a process' address space.
 - e.g., the base of the executable and position of the stack
 - Attacks:
 - Repetitively guess randomized address
 - Spraying injected attack code
- Windows has this enabled, software packages available for Linux and other UNIX variants

Buffer Overflow Defense

A simplified function stack (ignored stack frame pointer)!

- **Canary**

- Run-time stack check
- Push canary onto stack
- Canary value:
 - Constant 0x000aff0d
 - Or, random value decided during runtime (see my example)



Canary

- Supported by most platforms.
 - Minimal performance effects: 8% slow down for Web servers.
- Note: Canaries don't offer fool-proof protection.
 - Some attacks can leave canaries untouched.

Format String Vulnerability

- Who ever uses `printf` in their programs?

```
printf ("This class is %s\n", string);
```

- Or C++ Boost::format or C++ vsnprintf or C++ 2020 specs...

In some cases, `printf` or any functions use **format string** as inputs can be exploited

The diagram illustrates the mapping between the input arguments of a `printf` function and the corresponding format specifiers in the format string. The input line is: `printf("Color %s, Number %d, Float %4.2f", "red", 123456, 3.14);`. The output line is: `Color red, Number 123456, Float 3.14`. Arrows point from the format specifiers in the input string to the corresponding values in the output string: `%s` points to `red`, `%d` points to `123456`, and `%4.2f` points to `3.14`. Additionally, there are three long arrows pointing from the closing parentheses of the `printf` function call back to the opening parentheses, indicating the function's execution flow.

Input: `printf("Color %s, Number %d, Float %4.2f", "red", 123456, 3.14);`

Output: `Color red, Number 123456, Float 3.14`

Format String Vulnerability

- `printf` takes a format string and an arbitrary number of subsequent arguments
 - Format string determines what to print
 - Including a set of format parameters
 - Arguments supply input for format parameters
 - Which may be values (e.g., `%d`) or references (e.g., `%s`)
 - An argument for each format parameter

```
printf ("This class is %s\n", string);
```

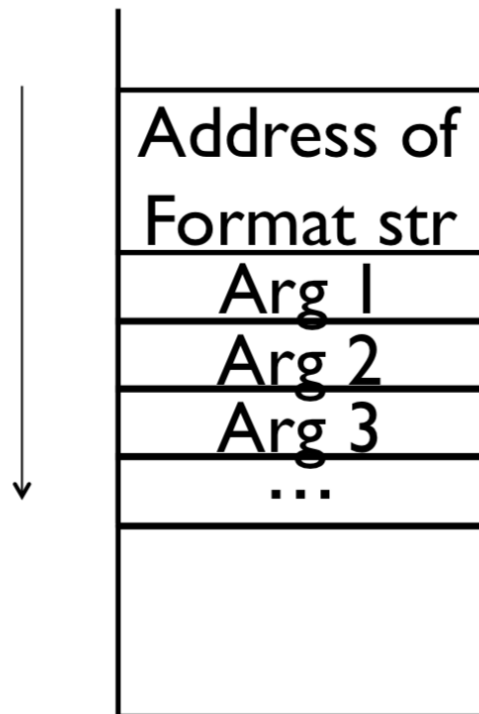
Format String Vulnerability

- Who uses `printf` in their programs?
 - In some cases, `printf` can be exploited
- As usual, arguments are retrieved from the stack
 - What happens when the following is done?

```
printf("%s%s%s%s");
```

- Traditionally, compilers **do not check** for a match between **arguments** and **format string**
 - So, `printf` would print “strings” using next four values on stack as string addresses – whatever they are

Printf and Stack



- Remember these are parameters to a function call
- So, the function expects them on the stack
- Printf will just start reading whatever is above the format string address

Format String Vulnerability

- printf can take a variable as an argument – treated as a format string
 - If **an adversary** can **control this argument**, they can direct printf to access that memory – “%s%s%s...”

You might be surprised but I see this on Twitter last summer....



Max
@_mxms

There's also a format string bug going the opposite direction (when your phones name is %p%p%p...)

[翻译推文](#)



Format String Vulnerability

- An interesting format parameter type – %n
 - “%n” in a format string tells the printf to write the number of bytes written via the format string processing up to that point to an address specified by the argument

```
#include <stdio.h>

int main()
{
    int val;

    printf("blah %n blah\n", &val);

    printf("val = %d\n", val);

    return 0;
}
```



```
blah  blah
val = 5
```

output

Enable arbitrary memory write by controlling the format string and the second argument.

Prevent Format String Vulnerability

- Preventing format string vulnerabilities means limiting the ability of adversaries to **control the format string**, how?
 - Hard-coded strings w/ no arguments – when you can
 - Hard-coded format strings at least – no `printf(arg)`
 - Do not use `%n`

SQL Injection

- **SQL injection:**

Causing undesired SQL queries to be run on your database.

- Often caused when untrusted input is pasted into a SQL query

PHP: `"SELECT * FROM Users WHERE name='$name'";`

- specify a user name of: **`x' OR 'a'='a`**

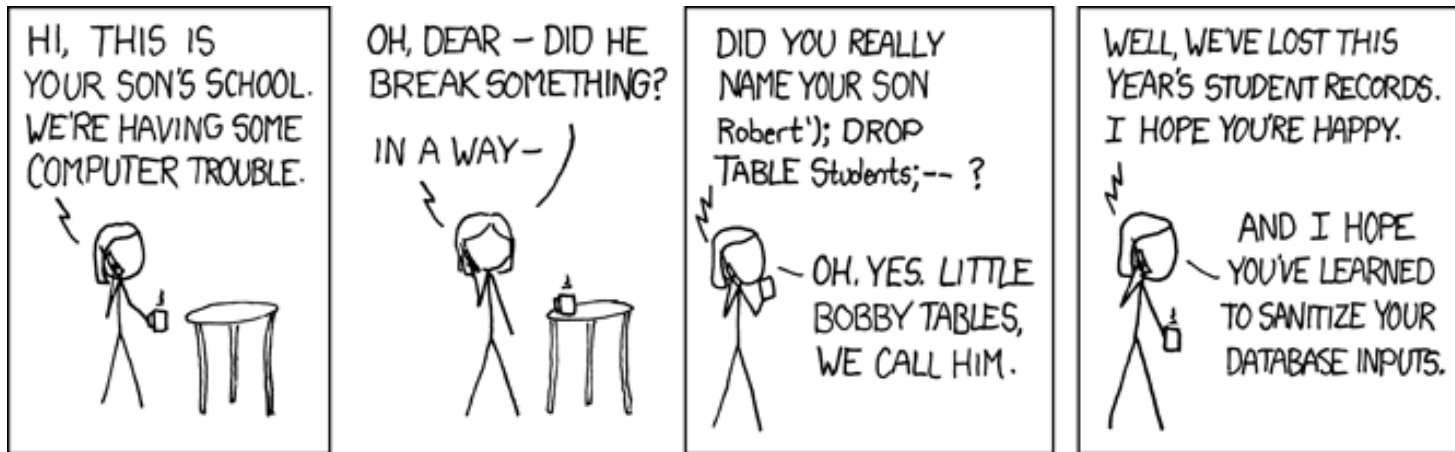
`SELECT * FROM Users WHERE name='x' OR 'a'='a';`

SQL Injection

- **SQL injection:**

Causing undesired SQL queries to be run on your database.

- Often caused when untrusted input is pasted into a SQL query



```
SELECT * FROM Users WHERE name=( 'Robert' ); DROP TABLE Students;
```

http://www.circleid.com/posts/20130325_sql_injection_in_the_wild/



Best defence

- If possible, use bound variables with prepared statements
 - Many libraries allow you to bind inputs to variables inside a SQL statement
 - PERL example (from <http://www.unixwiz.net/techtips/sql-injection.html>)

```
$sth = $dbh->prepare("SELECT email, userid FROM members WHERE  
email = ?;");
```

```
$sth->execute($email);
```

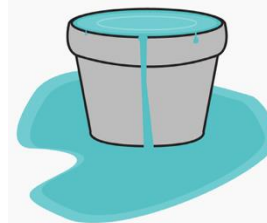
AI in Software Security

High-level principle and examples

How to find vulnerabilities in software?

```
#include <stdio.h>
#define MAX_IP_LENGTH 15
int main(void) {
    char file_name[] = "ip.txt";
    FILE *fp;
    fp = fopen(file_name, "r");
    char ch;
    int counter = 0;
    char buf[MAX_IP_LENGTH];
    while((ch = fgetc(fp)) != EOF) {
        buf[counter++] = ch;
    }
}
```

Buffer overflow if "ip.txt" has more than 15 bytes.



**BUFFER
OVERFLOW
ATTACKS**

Challenges in detecting overflows

- Non-trivial patterns:
 - Buffer overflows often involve subtle conditions and cross-function interactions.
- Manual audits are hard:
 - Uses manually defined rules/patterns.
 - Needs human reviews
- Dynamic testing limits:
 - Fuzzing (**we will talk about that next lecture**) can catch some buffer overflows by inducing crashes, but it requires carefully crafted inputs and may miss deep or state-dependent bugs.

Why traditional tools struggle?

- Limit of Static Analysis (SAST): Relies on rigid rules
- Problems:
 - High False Positives: Flagging safe code as vulnerable
 - High False Negatives: Missing complex bugs
- Traditional tools struggle to see the link in complex context

Why AI is a better fit?

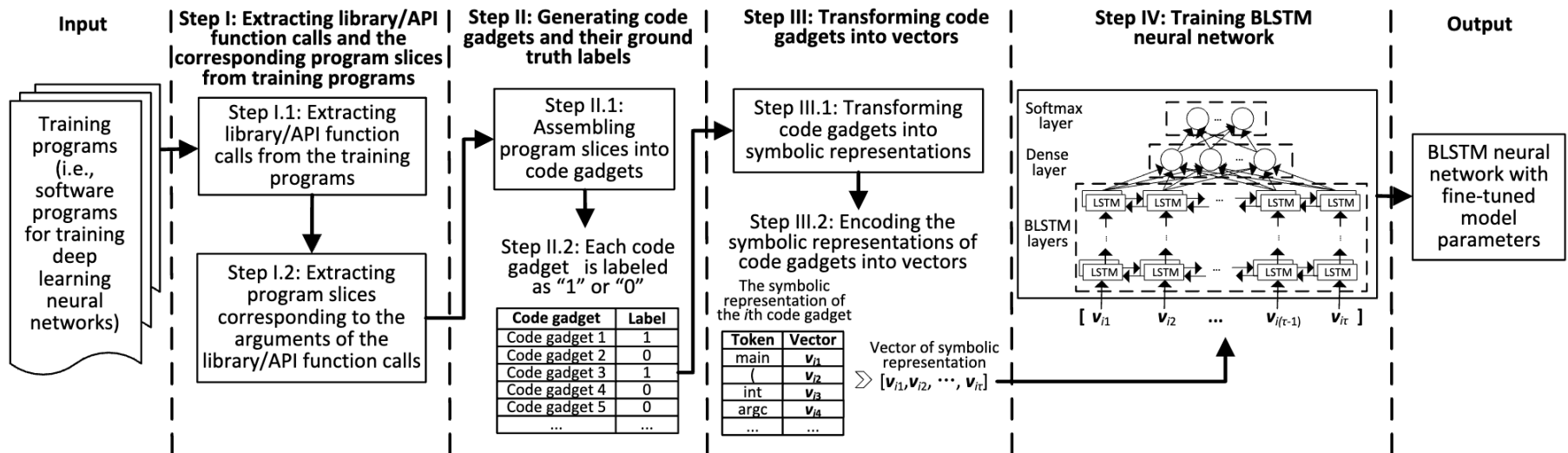
- Treat code as data
- Learn what vulnerable code looks like
- Catch variants that don't exactly match known patterns
- No need for hand-crafted rules, the model discovers relevant features

Recurrent Neural Networks (RNNs)

- Treat code as a sequence of words, like natural language
- Handle long context
- Capture ordering of operations, especially certain API call sequences
 - E.g., whether a buffer write operation is accompanied by a length check

Recurrent Neural Networks (RNNs)

LSTM-based API calls analysis



Graph Neural Networks (GNNs)

- Treat code as a graph
- Capture the structure of a program
- Graph Representations:
 - Control Flow Graphs (CFG)
 - Data Flow Graphs (DFG)
- Example: **Buffer Overflow Detection**
 - Capture complex state transitions and dynamic allocations

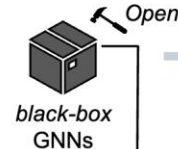
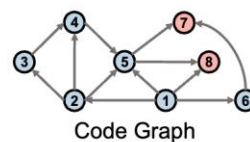
Graph Neural Networks (GNNs)

Buffer Overflow Detection

CVE-2016-10190

```
static int http_read_stream(  
    URLContext *h, uint8_t *buf, int size) ①  
{  
    HTTPContext *s = h->priv_data; ②  
    ...  
    if (s->chunksize >= 0) {  
        if (!s->chunksize) { ③  
            ...  
            s->chunksize = strtoll(line, NULL, 16); ④  
            ...  
        }  
        size = FFMIN(size, s->chunksize); ⑤  
    }  
    int len;  
    len = s->buf_end - s->buf_ptr; ⑥  
    if (len > 0) {  
        if (len > size) len = size;  
        memcpy(buf, s->buf_ptr, len); ⑦  
        ...  
    } else {  
        ...  
        len = ffurl_read(s->hd, buf, size); ⑧  
        ...  
    }  
}
```

Vulnerability Detection



Vulnerable

Why my written code is detected as vulnerable?

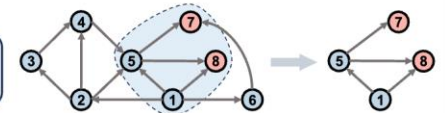


Generating Explanations

Factual Reasoning



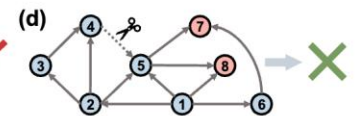
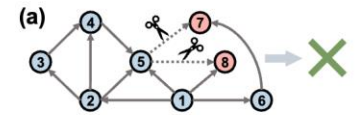
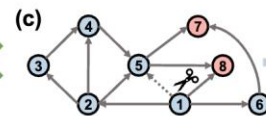
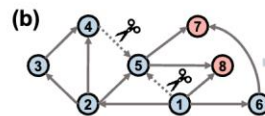
The selected sub-graph (①, ⑤, ⑦, and ⑧) is the key feature that contributed to the detected vulnerability.



Analysis of What-If



The **chunksize**, passed from ④, causes a miscalculation of **size** at ⑤, which in turn triggers the vulnerability at ⑦ and ⑧. Thus, please inspect the value of **chunksize** in ④.



Large Language Models (LLMs)

- Programming languages are like natural languages
- Pre-training data contains a large amount of code:
 - Learn syntax and semantics of programming languages
 - Generalized models: GPT, Gemini, Claude, ...
 - Specialized models: CodeBERT, GraphCodeBERT, ...
- Fine-tuned on vulnerability data
 - Learn patterns of vulnerable code

Large Language Models (LLMs)

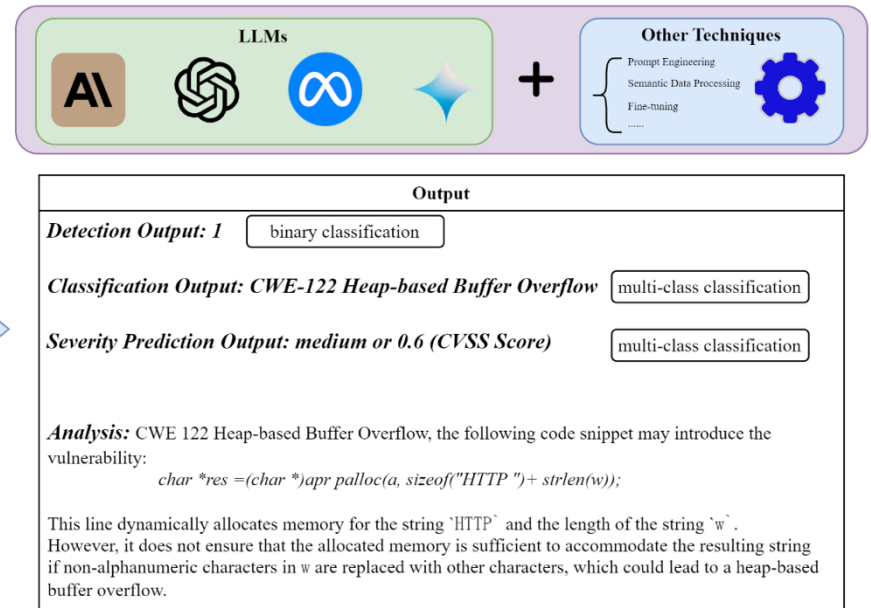
- Understand code and explain bugs like humans
- In addition to code patterns, it can also integrate naming and stylistic cues into its decision
 - E.g., a variable named `buffer_size` exists but is never used to limit a copy
- Require careful fine-tuning and prompt engineering

Large Language Models (LLMs)

```
1 static char *http2env(apr_pool_t *a, const char test_snippet)
2 {
3     char *res = (char *)apr_palloc(a, sizeof("HTTP_") + strlen(w));
4     char *cp = res;
5     char c;
6
7     *cp++ = 'H';
8     *cp++ = 'T';
9     *cp++ = 'T';
10    *cp++ = 'P';
11    *cp++ = '_';
12
13    while ((c = *w++) != 0) {
14        if (!apr_isalnum(c)) {
15            *cp++ = '_';
16        }
17        else {
18            *cp++ = apr_toupper(c);
19        }
20    }
21    *cp = 0;
22
23    return res;
24 }
```

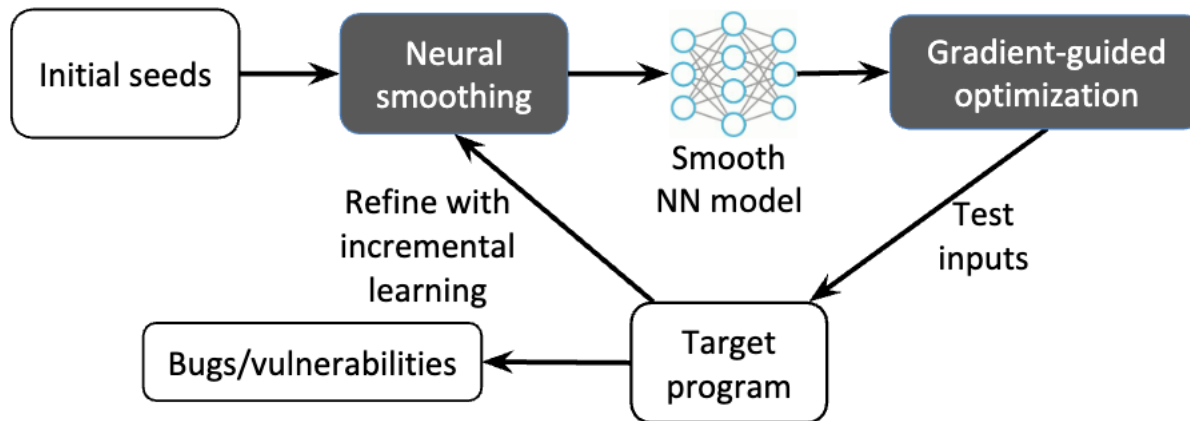


A sample code snippet from D2A dataset	
Vulnerable: 1	
CWE: CWE-122: Heap-based Buffer Overflow	
Severity: medium	



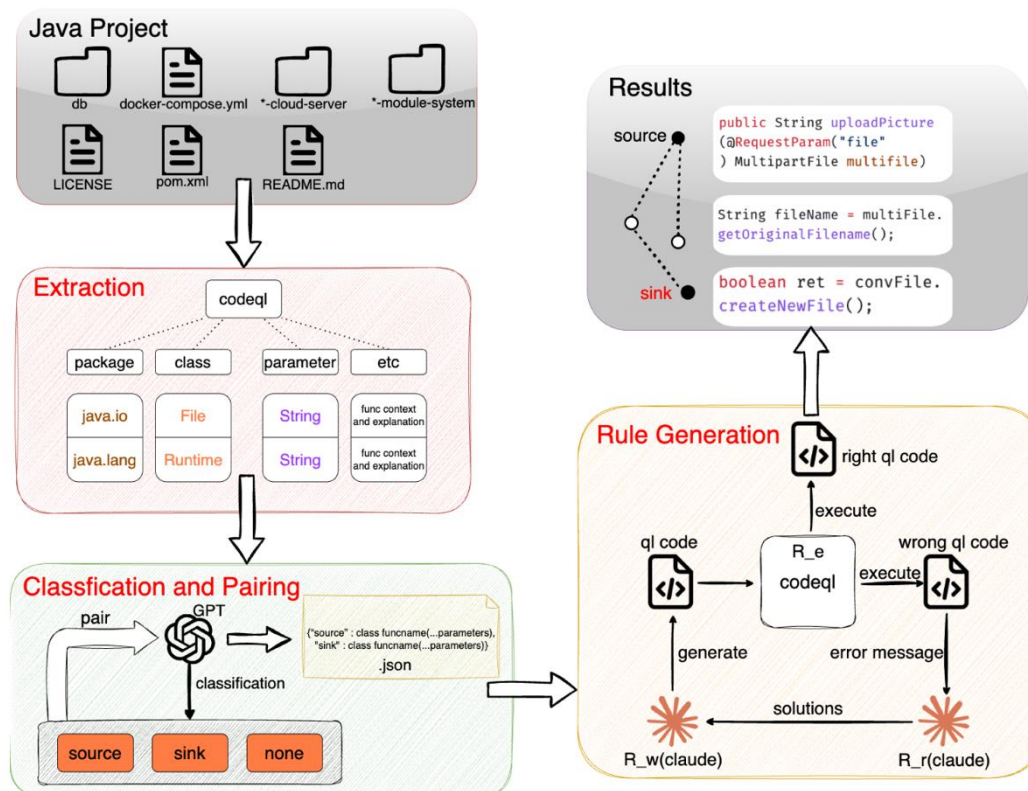
Traditional Tools with AI

AI-guided fuzzing: generate smart inputs



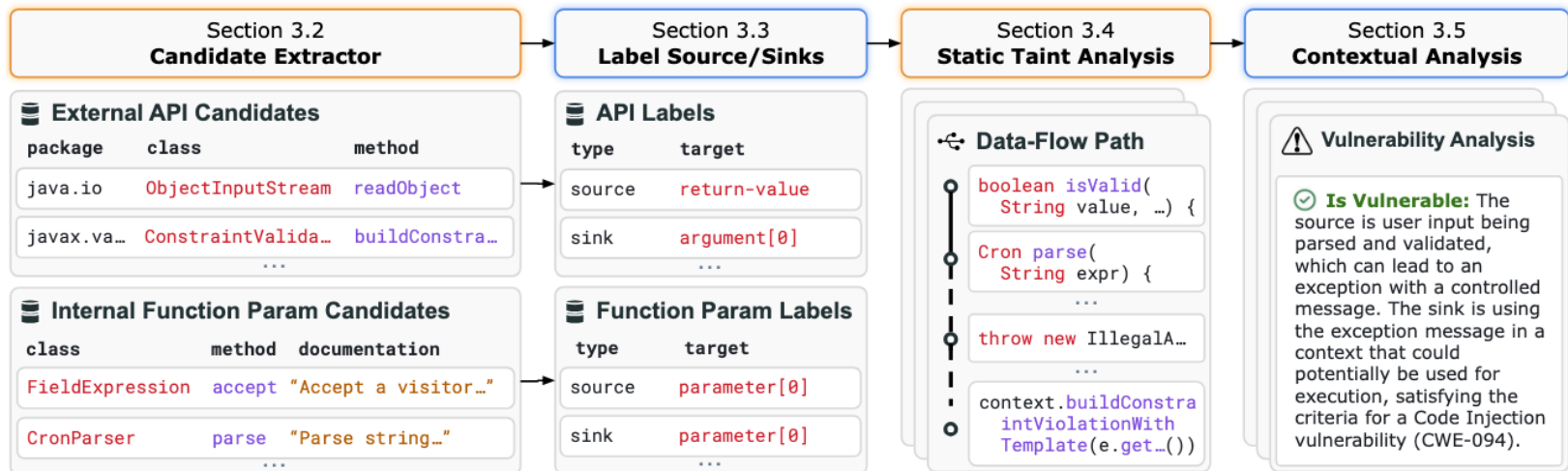
Traditional Tools with AI

LLM-assisted static analysis: generate rules for static analysis tools (like CodeQL)



Traditional Tools with AI

LLM-assisted static analysis: analyze and explain results from static analysis tools



From Detection to Repair

- Graph neural networks or language models
- Predict correct variables

```
1 def validate_sources(sources):  
2     object_name = get_content(sources, 'obj')  
3     subject_name = get_content(sources, 'subj')  
4     result = Result()  
5     result.objects.append(object_name)  
6     result.subjects.append(object_name)  
7     return result
```

```
1 def validate_sources(sources):  
2     object_name = get_content(sources, 'obj')  
3     subject_name = get_content(sources, 'subj')  
4     result = Result()  
5     result.objects.append(object_name)  
6     result.subjects.append(subject_name)  
7     return result
```

From Detection to Repair

- Seq2seq models
- “Translating” buggy code to fixed code
 - Like translating English to Chinese

Hypothetical buggy source code: (a)

```
int getVal(int *array, int len, int index) {  
  if (index < len)  
  {  
    return array[index];  
  } else {  
    return index;  
  }  
}
```

Hypothetical repaired function: (b)

```
int getVal(int *array, int len, int index) {  
  if (index < len && index >= 0)  
  {  
    return array[index];  
  } else {  
    return -1;  
  }  
}
```

Color key:

- Incorrect line
- Correct line
- Context before
- Context after
- VRepair special token

Neural network input (CWE-xxx, StartLoc and EndLoc tokens added, newlines removed):

(c)

CWE-119 int getVal (int * array , int len , int index) { <StartLoc> if (index < len) <EndLoc> { return array [index] ; } else { return index ; } }

Neural network token context diff output with context size 3:

(d)

<ModStart> index < len && index >= 0 <ModStart> else { return -1 <ModEnd> ; }
3 context tokens Tokens inserted after 'index < len' Token replaces 'index'

Syntax for 3 types of change modification

(f)

Type	Syntax
Add	<ModStart> context new
Delete	<ModStart> context <ModEnd> context
Replace	<ModStart> context new <ModEnd> context

Neural network token context diff output with context size 2:

(e)

<ModStart> < len && index >= 0 <ModStart> { return -1 <ModEnd> ; }

Context size 3 has only the correct repair as a possible interpretation, but context size 2 has these 3 interpretations:

(g)

```
int getVal ( int * array , int len , int index ) { if ( index < len && index >= 0 ) { return -1 ; } else { return index ; } }  
int getVal ( int * array , int len , int index ) { if ( index < len && index >= 0 ) { return -1 ; } }  
int getVal ( int * array , int len , int index ) { if ( index < len && index >= 0 ) { return array [ index ] ; } else { return -1 ; } }
```


Demo and Open Problems...

Problem: Every morning Aya goes for a 9 km long walk and stops at a coffee shop afterwards. When she walks at a constant speed of s kilometers per hour, the walk takes her 4 hours, including t minutes spent in the coffee shop. When she walks $s+2$ kilometers per hour, the walk takes her 2 hours and 24 minutes, including t minutes spent in the coffee shop. Suppose Aya walks at $s+\frac{1}{2}$ kilometers per hour. Find the number of minutes the walk takes her, including the t minutes spent in the coffee shop.

✗ Qwen2.5-32B-Instruct

To solve the problem, we need to determine the time Aya spends walking [omitted for brevity...]
The time spent walking is the distance divided by the speed, so the time spent walking is $(9/s)$ hours. Therefore, the equation for the total time is: $9/s + t = 240$, where t is the time spent in the coffee shop in minutes.
Next, we know that [omitted for brevity...]. The time spent walking is $9/(s+2)$ hours. Therefore, the equation for the total time is: $9/(s+2) + t = 144$.
We now have two equations:
1. $9/s + t = 240$
2. $9/(s+2) + t = 144$
[omitted for brevity...]
So, $s = 3/16$ km/h. Now we substitute $s = 3/16$ km/h back into equation (1) to find t :
 $9/(3/16) + t = 240$, $48 + t = 240$, $t = 192$
[omitted for brevity...]
So, the number of minutes the walk takes her, including the t minutes spent in the coffee shop, is 205.

unable to correct false statements above

Did not cross verify equation 2

✓ DeepSeek-R1

Okay, let's try to figure out this problem step by step.
So, Aya goes for a 9-kilometer walk every morning and then stops at a coffee shop. [omitted for brevity...]
First, let's parse the problem. Let's denote the walking [omitted for brevity...]. Our goal is to find the total time when she walks at $s+1/2$ km/h. Hmm, so we need to figure out the values of s and t first.
Let me write down the equations based on the given information. [omitted for brevity...]
Now we have two equations:
1) $9/s + t/60 = 4$, 2) $9/(s+2) + t/60 = 2.4$
We can subtract the second equation from the first to eliminate [omitted for brevity...]
So the left side becomes: [omitted for brevity...]
Multiply both sides by: [omitted for brevity...]
Now, solving this quadratic equation: [omitted for brevity...]. So, Aya's original speed is 2.5 km/h.
Now, let's find t [omitted for brevity...]. Wait, let me confirm. But wait, the question says [omitted for brevity...]. Therefore, the answer should be 204 minutes.
But let me just check again to make sure I didn't make any mistakes in the quadratic equation solving. Let's verify that $s = 2.5$ is correct. [omitted for brevity...]. Thus, the answer should be 204.

think with anthropomorphic tone

detailed steps in calculation

self-reflection

✓ LIMO

Okay, let's see. So Aya goes for a 9-km walk every morning and then stops at a coffee shop. The time she takes for the walk, including the time in the coffee shop, depends on her walking speed. [omitted for brevity...]. When she walks a bit faster, at s plus 2 kilometers per hour, the total time is 2 hours and 24 minutes, which is 2 hours and 24 minutes, so that's 2.4 hours?
Wait, 24 minutes is 0.4 hours? Wait, no. Wait, 60 minutes is 1 hour, so 24 minutes is 24/60, which is 0.4 hours. So 2 hours and 24 minutes is 2.4 hours. [omitted for brevity...]
So, let's set up equations. Let's denote the walking time as T_1 when she walks at s km/h, and T_2 when she walks at $s+2$ km/h. Then, $T_1 + t = 4$ hours, and $T_2 + t = 2.4$ hours. **But t is in minutes, so we need to convert that to hours or convert the total time to minutes. Maybe it's better to keep everything in hours.** So t minutes is $t/60$ hours. [omitted for brevity...]
For the first case: walking time $T_1 = 9/s$ hours, then total time $T_1 + t/60 = 4$ hours.
For the second case: walking time $T_2 = 9/(s+2)$ hours, then total time $T_2 + t/60 = 2.4$ hours. So now we have two equations: 1) $9/s + t/60 = 4$, 2) $9/(s+2) + t/60 = 2.4$
Let me write that as:
 $9*(1/s - 1/(s+2)) = 1.6$. So, $1/s - 1/(s+2) = 1.6/9$
Let me compute 1.6 divided by 9. 1.6 divided by 9 is 0.1777... or 16/90, which simplifies to 8/45. So $1.6/9 = 8/45 \approx 0.1777...$ So, $1/s - 1/(s+2) = 8/45$
Now, let's compute the left side: [omitted for brevity...]. Multiply both sides by 2: [omitted for brevity...]. Let me write that as a quadratic equation: [omitted for brevity...]. Now, let's solve this quadratic equation: [omitted for brevity...]. So, s is 2.5 km/h.
Let's verify that. If s is 2.5 km/h, then walking 9 km would take $9/2.5 = 3.6$ hour [omitted for brevity...]
But let me check again. Maybe I made a mistake in calculations. First, $s = 2.5$ km/h. Then, walking time at s is $9/2.5 = 3.6$ hours... [omitted for brevity...]. **Alternatively, we could have converted everything to minutes from the start.** Let's see: First walk: $9/(s)$ hours + t minutes = 4 hours. 4 hours is 240 minutes. [omitted for brevity...]. I think that's solid. So the answer is 204.

Crafting high-quality Chain-of-Thought data is very hard in software security...